

Single-Task and Multi-Task Soft Actor Critic on Meta-World Benchmarks

Erik Bonek
TU Wien
Vienna, Austria
erik.bonek@tuwien.ac.at
e12118550

Philipp Jelinek
TU Wien
Vienna, Austria
philipp.jelinek@tuwien.ac.at
e12106342

Philipp Haimerl
TU Wien
Vienna, Austria
philipp.haimerl@tuwien.ac.at
e12102566

Andreas Michael Kohler
TU Wien
Vienna, Austria
andreas.kohler@tuwien.ac.at
e12102563

Abstract—Deep Reinforcement Learning has shown good results in single task decision making problems. Multi-Task reinforcement learning is an active field of research with the goal of generalization by training a single agent across multiple tasks. A systematic empirical analysis of the SAC-algorithm and well known multi-task enhancements is conducted. It is shown that incorporating one-hot task encoding provides the most significant performance improvement for the Meta-World MT3 benchmark and using failure-rate-based curriculum learning together with disentangled alphas for the MT10 benchmark.

Index Terms—Multi-task, Reinforcement learning, Robotics, Manipulation, Curriculum Learning

I. INTRODUCTION

Deep Reinforcement Learning (DRL) has shown good results in various decision making problems such as playing Atari [1], Go [2], but also in the field of robotics [3]–[5]. While these successes demonstrate the potential of DRL, most approaches are commonly evaluated on a single task. Multi-Task Reinforcement Learning (MTRL) is an active research topic and has been studied as a means to improve data efficiency and generalization by training a single agent across multiple tasks [6].

In this work, no new multi-task reinforcement learning algorithm is introduced. Instead, a systematic empirical analysis is conducted to examine how different established extensions to a standard off-policy reinforcement learning algorithm influence multi-task learning performance in manipulation tasks. Soft-Actor-Critic (SAC) [7] is used as a common baseline, and several adopted multi-task enhancements such as

- One-hot encoding,
- Disentangled Alphas (DA),
- Project conflicting Gradients (PCGrad),
- Curriculum Learning (CL), and
- Multi-head architecture for the Actor as well as the Critics

are compared.

For the experiments the Meta-World benchmark [8] is used, focusing on a custom benchmark, MT3, which is composed of the three tasks

- reach-v3,
- push-v3, and
- pick-place-v3,

as well as the multi-task benchmark MT10. Performance is evaluated in terms of sample efficiency, training stability, and average task return, in order to provide a structured comparison of multi-task Soft-Actor-Critic (MT-SAC) extensions.

II. THEORETICAL BACKGROUND AND ALGORITHMS

The SAC-algorithm [7] is used as a common baseline. It is an off-policy, model-free reinforcement learning algorithm capable of handling continuous action spaces. It combines high sample efficiency achieved through the use of a replay buffer, with entropy-based exploration, ensuring effective learning and robust policy performance. The goal of standard Reinforcement Learning (RL) is to maximize the expected sum of rewards

$$\sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t)] \quad . \quad (1)$$

For SAC a more general maximum entropy objective is used which encourages exploration of actions that were not visited previously. With the learnable temperature parameter α , Equation (1) becomes

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))] \quad , \quad (2)$$

with $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s_t)] \quad ,$

see [7].

Based on [9] and initial results, we have chosen large networks in our experiments for the actor and critic in order to maximize the achieved success rate. The exact parameter setup for all of our experiments can be found in Tables V to XI. For

training, we employed the PyTorch-based implementation provided by the Stable-Baselines3 reinforcement learning library [10].

A. One-Hot Encoding

One-hot encoding refers to a method where categorical variables, such as which task is currently trained on, are transformed into binary information. Given a set $\mathcal{B}$ of tasks with the cardinality of $|\mathcal{B}| = N$, one-hot encoding creates an N -dimensional feature vector v_i , which can be appended to the observation space. Each vector is defined as

$$v_i[j] = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

This additional information allows the agent to distinguish between different tasks, enabling task-specific behaviour in multi-task scenarios. All following SAC extensions are using one-hot encoding, as each extension requires knowledge about which task is currently trained.

B. Disentangled Alphas

Building on the one-hot task encoding, we employ a standard extension of SAC in which the entropy temperature is disentangled across tasks, [11]–[13]. While standard SAC uses a single global entropy temperature α to regulate the exploration/exploitation trade-off, this variant learns a separate entropy temperature for each task. The entropy-regularized objective for a given task i is given by

$$J_i(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha_i \mathcal{H}(\pi(\cdot | s_t))] \quad (4)$$

All other components of SAC, including the actor, double Q-value critics, replay buffer, and target network updates, remain unchanged. The task-conditioned entropy coefficient α_i is used consistently in both the critic target computation and the actor update.

C. Project Conflicting Gradients

A primary challenge in multi-task learning is the optimization landscape described by Yu et al. [14] as the “tragic triad”: the co-occurrence of high curvature, large discrepancies in gradient magnitudes, and conflicting gradients. Gradients are considered conflicting when they point in opposing directions, defined mathematically as having a negative cosine similarity, meaning a negative inner product ($\mathbf{g}_i \cdot \mathbf{g}_j < 0$). Under these conditions, a gradient update that benefits one task may be detrimental to another, leading to “destructive interference” [14].

To resolve this, we employ the PCGrad method. PCGrad modifies the gradients directly during the optimization step: for any pair of tasks $\mathcal{T}_i$ and $\mathcal{T}_j$, if their gradients $\mathbf{g}_i$ and $\mathbf{g}_j$ conflict, $\mathbf{g}_i$ is projected onto the normal plane of $\mathbf{g}_j$ and vice versa, shown in Figure 1. This removes the conflicting component while preserving the direction helpful for the joint objective. The projection update is given by:

$$\mathbf{g}_i = \mathbf{g}_i - \frac{\mathbf{g}_i \cdot \mathbf{g}_j}{\|\mathbf{g}_j\|^2} \mathbf{g}_j, \quad \text{if } \mathbf{g}_i \cdot \mathbf{g}_j < 0 \quad (5)$$

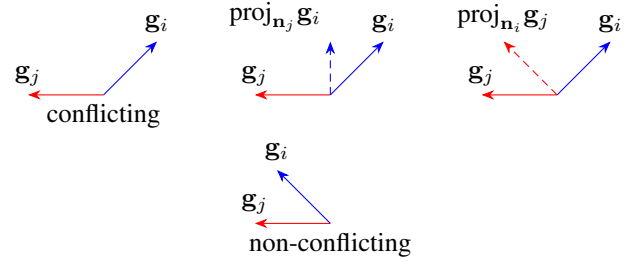


Fig. 1: Policy gradient projection visualized based on conflicting/non-conflicting task gradients.

The procedure is applied to all tasks in the batch. To ensure the process is symmetric and not biased towards the order of tasks, the sequence of comparisons is shuffled randomly at every update step [14]. The final update applied to the policy is the sum of these de-conflicted gradients.

D. Multi-Head Architectures

We evaluate two distinct architectural configurations to address task interference: a Multi-Head-Critic with a shared actor, and a multi-head architecture applied to both the actor and critic, the Multi-Head-Actor-Critic.

The multi-head architecture employs “Output-Level Routing”, formally defined by He et al. [15], to the networks. Output-level routing is the standard multi-head approach where tasks share a backbone but possess specific head modules which are chosen based on the active task.

Standard SAC implementations for multi-task learning often rely on a shared network architecture where the task index is provided via a one-hot encoding. However, as analysed in [14], sharing the full network parameters leads to conflicting gradient updates, where tasks with high-magnitude gradients can destabilize the value estimates of others.

While PCGrad [14] solves this problem, another approach to mitigate this is to employ the Multi-Head-Critic architecture. In this setup, the network consists of a shared encoder f_θ , the so-called “backbone”, which extracts features common to all tasks, while the Q-value estimation is routed to N independent output heads, as shown in Figure 2. For a given task k , the Q-value is calculated as

$$Q(s, a, k) = h_{\phi_k}(f_\theta(s, a)) \quad (6)$$

where f_θ denotes the shared encoder with parameters θ , and h_{ϕ_k} represents the task-specific head network parametrized by ϕ_k . This separation ensures that the value function for each task is learned independently after the feature extraction stage, preventing destructive interference in the final layers as suggested by recent applications in robotic control [16].

While some approaches like [16] utilize a shared actor with only the critic having a multi-head architecture, we extended this architecture to the actor network, similar to the multi-head architecture used in [14].

We define our policy as:

$$\pi(a | s, k) = \mathcal{N}(\mu_{\omega_k}(f_\theta(s)), \sigma_{\omega_k}(f_\theta(s))) \quad (7)$$

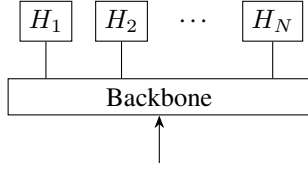


Fig. 2: Diagram of the Multi-Head-Critic architecture.

Here, μ_{ω_k} and σ_{ω_k} denote the distinct linear layers (parametrized by ω_k) that output the mean and standard deviation for task k . By using separate actor heads, we ensure that the policy distribution is parametrized independently, avoiding the “fighting” for capacity that occurs in a shared policy head.

E. Curriculum Learning

In multi-task RL, it is common that the robot is able to learn simpler tasks more quickly than difficult tasks. With curriculum learning, the robot starts learning on a reduced set of simple tasks, and once it has achieved good results on those tasks more difficult tasks are introduced to the robot [17].

In standard curriculum learning, the curriculum is chosen by a human. While humans might have some intuition which tasks may be difficult for the robot, this intuition can be wrong. Additionally, the creation of a curriculum by a human can be time-intensive, especially when a task is swapped for another one, as this may require the reordering of the curriculum. In this work, we explore an approach for dynamically creating the curriculum based on the failure rate of each task the robot has to learn.

A training environment where each task is chosen according to the corresponding task probability p_i is used. Initially, all N tasks are equally likely, so $p_i = \frac{1}{N}$. For each evaluation during training, the success rate s_i of each task is computed. To ensure a smooth transition of the task probabilities, the mean of the last M success rates is calculated, $\bar{s}_i$. For each task, the mean failure rate $\bar{f}_i = 1 - \bar{s}_i$ is computed. Finally, the task probabilities p_i are derived as

$$p_i = \frac{\bar{f}_i}{\sum_{i=1}^N \bar{f}_i} . \quad (8)$$

By using the failure rate to compute the task probabilities, once the robot learns to successfully complete a task, the task’s probability is reduced. Logically speaking, once the robot has learned the task, it doesn’t need to spend more time learning it. Instead, it can focus on gathering experience on more difficult tasks.

III. RESULTS

The individual methods are compared against each other based on

- performance in it/s,
- mean reward of all tasks,
- standard deviation, as well as
- the success rate of the model with highest reward.

A. Single Tasks

A 100% success rate was achieved for all three single tasks (reach, push, pick-place) using the SAC-algorithm with minimal hyperparameter tuning. This indicates that SAC can reliably learn task-specific policies, although the three selected tasks are relatively simple and do not represent the most challenging scenarios.

B. MT3

As can be seen in Table III, the standard SAC-algorithm is not able to learn a well-working generalized policy across all three tasks. One-hot task encoding, the baseline for all the other methods, provides the biggest improvement, while further additions do not lead to significant additional gains. While PCGrad and Multi-Head-Actor-Critic are able to outperform one-hot encoding after 3 million timesteps, they are not able to keep this lead, as shown in Figure 3a and Figure 3c. This results in a similar mean reward, standard deviation and success rate of the best model, seen in Table II and Table III, while taking significantly longer, indicated by the difference of it/s in Table I. The Multi-Head-Critic implementation has shown no improvement compared to one-hot task encoding, not only taking longer to train but also not able to outperform the algorithm at any moment by a significant margin. It should be noted that while the multi-head implementations have similar results to the baseline, their networks for the actor and critics have approximately 30% fewer weights than the ones of the baseline. The possible reduction in training time from reducing the network sizes is cancelled out by an increased overhead introduced by the required branching in the forward-pass of the multi-head architecture.

As shown in Figure 3b the methods CL and DA are not able to outperform the one-hot encoding implementation. The use of learning individual entropy coefficients for each task leads to a degradation of performance, especially for the pick-place task where 0% success rate is achieved. The reason for that is not immediately evident from the observed reward values. One possible explanation is that the model converges to an entropy coefficient that induces insufficient exploration for the pick and place task, thereby preventing the discovery of a successful policy. This hypothesis is supported by the learned entropy coefficients at the end of training for MT3, which are 0.032 for reach, 0.164 for push, and 0.0035 for pick-and-place, indicating substantially reduced exploration for the latter task although there is no suitable policy learned for that task. However, the combination of the CL and DA methods is the only one capable to achieve a success rate of 100%.

C. MT10

In contrast to MT3, the results of MT10 show different behaviour. As shown in Table IV the addition of one-hot task encoding alone is not enough for the algorithm to learn all tasks reliably. While multi-head and PCGrad extensions lead to slight improvement, their impact is limited. By using DA and the proposed curriculum learning approach, bigger performance gains are achieved.

TABLE I: Iterations per second during learning

Benchmark \ Method	Base	One hot Encoding	PCGrad	Disentangled Alphas	Curriculum Learning	Multihead		CL + DA
						Critic	Actor-Critic	
MT3	800 it/s	720 it/s	300 it/s	725 it/s	690 it/s	285 it/s	220 it/s	650 it/s
MT10	710 it/s	740 it/s	70 it/s	710 it/s	630 it/s	140 it/s	100 it/s	710 it/s

TABLE II: Mean Reward and standard deviation for the MT3 and MT10 benchmark

Benchmark \ Method	Base	One hot Encoding	PCGrad	Disentangled Alphas	Curriculum Learning	Multihead		CL + DA
						Critic	Actor-Critic	
MT3	1,137 ± 811	1,652 ± 207	1,658 ± 261	1,209 ± 854	1,576 ± 436	1,643 ± 230	1,696 ± 162	1,685 ± 176
MT10	1,196 ± 658	1,188 ± 660	1,244 ± 663	1,476 ± 347	1,350 ± 566	1,263 ± 653	1,286 ± 669	1,585 ± 199

TABLE III: Success Rate for the Best Model of the MT3 benchmark

Task \ Method	Base	One hot Encoding	PCGrad	Disentangled Alphas	Curriculum Learning	Multihead		CL + DA
						Critic	Actor-Critic	
mean	57 %	97 %	93 %	67 %	93 %	93 %	97 %	100 %
reach-v3	70 %	100 %	80 %	100 %	90 %	80 %	90 %	100 %
push-v3	100 %	90 %	100 %	100 %	100 %	100 %	100 %	100 %
pick-place-v3	0 %	100 %	100 %	0 %	90 %	100 %	100 %	100 %

TABLE IV: Success Rate for the Best Model of the MT10 benchmark

Task \ Method	Base	One hot Encoding	PCGrad	Disentangled Alphas	Curriculum Learning	Multihead		CL + DA
						Critic	Actor-Critic	
mean	67 %	69 %	70 %	89 %	85 %	75 %	77 %	98 %
reach-v3	80 %	80 %	100 %	100 %	100 %	100 %	100 %	100 %
push-v3	90 %	90 %	100 %	100 %	80 %	50 %	70 %	80 %
pick-place-v3	0 %	0 %	0 %	100 %	80 %	0 %	0 %	100 %
door-open-v3	100 %	90 %	100 %	100 %	100 %	100 %	100 %	100 %
drawer-open-v3	0 %	30 %	0 %	0 %	100 %	100 %	100 %	100 %
drawer-close-v3	100 %	100 %	100 %	100 %	100 %	100 %	100 %	100 %
button-press-topdown-v3	100 %	100 %	100 %	100 %	100 %	100 %	100 %	100 %
peg-insert-side-v3	0 %	0 %	0 %	90 %	0 %	0 %	0 %	100 %
window-open-v3	100 %	100 %	100 %	100 %	90 %	100 %	100 %	100 %
window-close-v3	100 %	100 %	100 %	100 %	100 %	100 %	100 %	100 %

The effect of curriculum learning is illustrated in Figure 4b where increases in reward can always be observed whenever a new task is successfully learned. In contrast, after an initial phase of fast learning, most extensions show a flattening of the reward curve at around 4 million timesteps, indicating limited training improvement. Learning with disentangled alphas, however, enables the network to continuously improve over time, as reflected with a constant upwards trend of the reward curve. This suggests that disentangling the entropy coefficient helps balance exploration and exploitation differently across tasks, which becomes increasingly important as task diversity grows. At some point, one task is already completely learned and doesn't need further exploration, while another task doesn't have a good policy yet. With different alphas each task can explore as much as it needs.

Combining curriculum learning and task disentangled entropy coefficients results in the best overall performance, both in terms of reward as seen in Figure 4b and the final success rate in Table IV, while also being able to match the training speed of one-hot task encoding, Table I. The achieved success

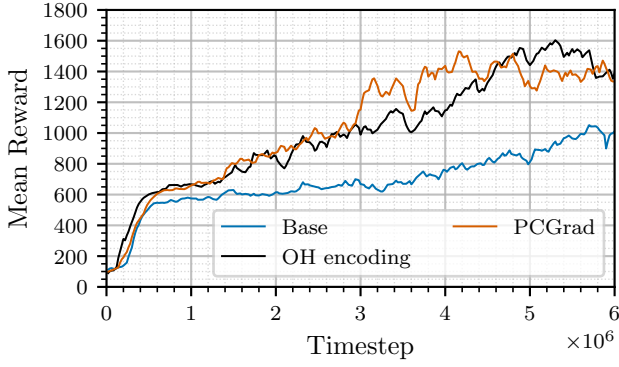
rate of 98 % exceeds that of all other evaluated methods in this study and highlights the effectiveness of the proposed combination for more complex multi-learning tasks.

IV. CONCLUSION

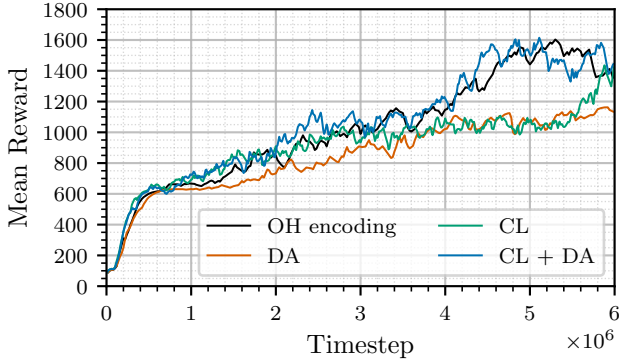
This work investigated the application of SAC for multi-task reinforcement learning and evaluated several established extensions alongside a curriculum learning approach based on the failure rates of the different tasks. While the base SAC achieves a success rate of 100 % for all single tasks, its ability to learn generalized policies without any task-specific extensions is limited.

For the MT3 benchmark, one-hot encoding was sufficient to achieve strong performance, showing that the tasks are sufficiently similar, so that one single network can learn a shared policy. More sophisticated approaches could not yield any meaningful improvements, especially because one-hot encoding already yields a 97 % success rate.

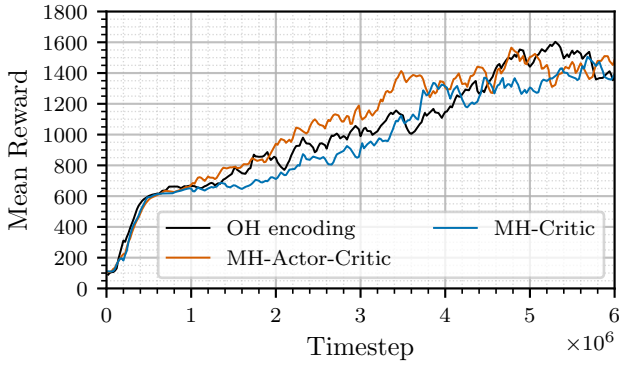
In contrast, MT10 represents a significantly more challenging setting, where one-hot encoding alone is not enough.



(a)



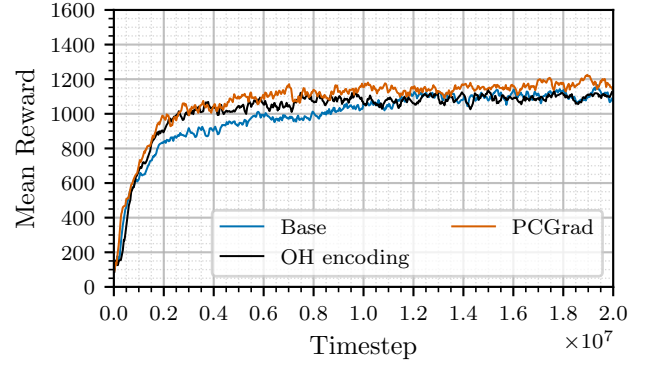
(b)



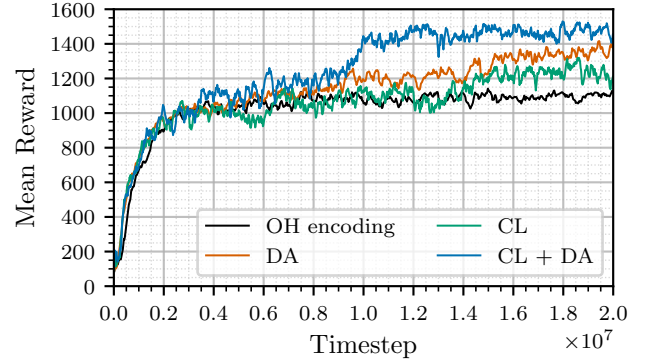
(c)

Fig. 3: Average reward during learning of different algorithms on the MT3 benchmark.

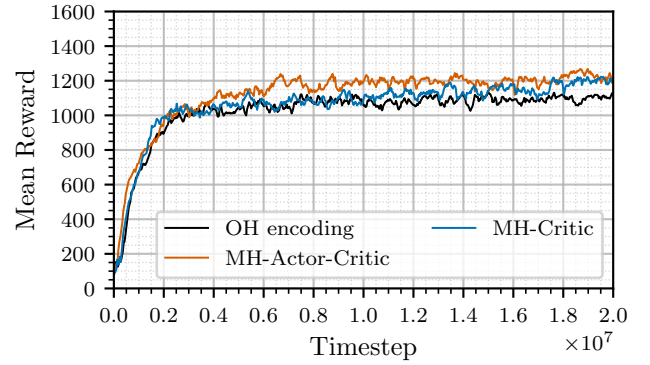
Substantial performance gains were achieved through the use of disentangled alpha and the proposed curriculum learning approach. The combination of curriculum learning and task-specific entropy resulted in the best overall performance with a 98% success rate on MT10 while maintaining competitive training speeds. These results highlight the importance of adaptive exploration and intelligent task-choosing for curriculum learning.



(a)



(b)



(c)

Fig. 4: Average reward during learning of different algorithms on the MT10 benchmark.

We assume that the negligible performance impacts of the multi-head architectures and PCGrad stem from the fact that our networks are already large enough to accommodate the induced gradients by the different tasks without suffering from significant destructive interference. This suggests that the baseline model already possesses sufficient capacity to learn disjoint or compatible representations for multiple tasks, rendering the explicit conflict-mitigation mechanisms of PCGrad

and the hard parameter separation of multi-head architectures largely redundant in this setting.

Future work could investigate the difference in learning success for the different evaluated methods for more resource constraint networks. Additionally, extending the proposed approach of curriculum learning with disentangled alpha to larger benchmarks such as MT50 or real-world robotic systems could be explored next.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing atari with deep reinforcement learning," *ArXiv*, vol. abs/1312.5602, 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15238391>
- [2] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017. [Online]. Available: <https://doi.org/10.1038/nature24270>
- [3] H. Das, M. N. Vu, and C. Ott, "Learning swing-up maneuvers for a suspended aerial manipulation platform in a hierarchical control framework," 2025. [Online]. Available: <https://arxiv.org/abs/2506.13478>
- [4] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, "Real-time neural mpc: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, p. 2397–2404, Apr. 2023. [Online]. Available: <http://dx.doi.org/10.1109/LRA.2023.3246839>
- [5] P. Kormushev, S. Calinon, and D. G. Caldwell, "Reinforcement learning in robotics: Applications and real-world challenges," *Robotics*, vol. 2, no. 3, pp. 122–148, 2013. [Online]. Available: <https://www.mdpi.com/2218-6581/2/3/122>
- [6] R. Caruana, "Multitask learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997. [Online]. Available: <https://doi.org/10.1023/A:1007379606734>
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *CoRR*, vol. abs/1801.01290, 2018. [Online]. Available: <http://arxiv.org/abs/1801.01290>
- [8] R. McLean, E. Chatzaroulas, L. McCutcheon, F. Röder, T. Yu, Z. He, K. Zentner, R. Julian, J. K. Terry, I. Woungang, N. Farsad, and P. S. Castro, "Meta-world+: An improved, standardized, RL benchmark," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. [Online]. Available: <https://openreview.net/forum?id=1de3azE606>
- [9] R. McLean, E. Chatzaroulas, J. Terry, I. Woungang, N. Farsad, and P. S. Castro, "Multi-task reinforcement learning enables parameter scaling," 2025. [Online]. Available: <https://arxiv.org/abs/2503.05126>
- [10] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [11] T. garage contributors, "Garage: A toolkit for reproducible reinforcement learning research," <https://github.com/rlworkgroup/garage>, 2019.
- [12] V. Joshi, Z. Xu, B. Liu, P. Stone, and A. Zhang, "Benchmarking massively parallelized multi-task reinforcement learning for robotics tasks," 2025. [Online]. Available: <https://arxiv.org/abs/2507.23172>
- [13] J. He, K. Li, Y. Zang, H. Fu, Q. Fu, J. Xing, and J. Cheng, "Efficient multi-task reinforcement learning with cross-task policy guidance," 2025. [Online]. Available: <https://arxiv.org/abs/2507.06615>
- [14] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient surgery for multi-task learning," *CoRR*, vol. abs/2001.06782, 2020. [Online]. Available: <https://arxiv.org/abs/2001.06782>
- [15] J. He, K. Li, Y. Zang, H. Fu, Q. Fu, J. Xing, and J. Cheng, "Not all tasks are equally difficult: Multi-task deep reinforcement learning with dynamic depth routing," 2024. [Online]. Available: <https://arxiv.org/abs/2312.14472>
- [16] Y. Fan, Z. Pei, H. Shi, M. Li, T. Guo, and Z. Tang, "Quadruped robot locomotion via soft actor-critic with multi-head critic and dynamic policy gradient," *Applied Intelligence*, vol. 55, no. 10, p. 720, 2025. [Online]. Available: <https://doi.org/10.1007/s10489-025-06584-1>
- [17] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, ser. ICML '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 41–48. [Online]. Available: <https://doi.org/10.1145/1553374.1553380>

APPENDIX

TABLE V: Training parameters for reach task

Parameter Name	Parameter Value
max_episode_steps	500
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	3e-4
buffer_size	1 000 000
batch_size	256
tau	5e-3
gamma	0.99
gradient_steps	-1
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	[256, 256, 256]
log_std_init	-3
use_expln	False

TABLE VI: Training parameters for push task

Parameter Name	Parameter Value
max_episode_steps	250
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	4e-4
buffer_size	500 000
batch_size	512
train_freq	1
tau	5e-3
gamma	0.99
gradient_steps	-1
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	[256, 256, 256]
log_std_init	-3
use_expln	False

TABLE VII: Training parameters for pick-place task

Parameter Name	Parameter Value
max_episode_steps	1000
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	5e-4
buffer_size	800 000
batch_size	1024
train_freq	3
tau	5e-3
gamma	0.95
gradient_steps	6
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	[1024, 1024, 512]
log_std_init	-3
use_expln	False

TABLE VIII: Hyperparameters for Base, Disentangled Alpha, One Hot Encoding PC Grad for MT3 and M10 multi-task

Parameter Name	Parameter Value
max_episode_steps	200
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	3e-4
buffer_size	300 000
batch_size	128
train_freq	1
tau	1e-3
gamma	0.99
gradient_steps	1
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	[512, 1024, 1024, 512]
log_std_init	-3
use_expln	False

TABLE IX: Hyperparameters for Curriculum Learning and Curriculum Learning with Disentangled Alpha for MT3 and M10 multi-task

Parameter Name	Parameter Value
max_episode_steps	200
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	3e-4
buffer_size	300 000
batch_size	512
averaging_success_window	10
train_freq	1
tau	1e-3
gamma	0.99
gradient_steps	1
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	[512, 1024, 1024, 512]
log_std_init	-3
use_expln	False

TABLE X: Hyperparameters for Multihead-Actor-Critic for MT3 and M10 multi-task

Parameter Name	Parameter Value
max_episode_steps	200
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	3e-4
buffer_size	300 000
batch_size	512
averaging_success_window	10
train_freq	1
tau	1e-3
gamma	0.99
gradient_steps	1
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	actor and critic backbone: [512, 1024, 512], actor and critic heads: [512, 256]
log_std_init	-3
use_expln	False

TABLE XI: Hyperparameters for Multihead-Critic for MT3 and M10 multi-task

Parameter Name	Parameter Value
max_episode_steps	200
seed	42
timesteps	2 000 000
policy	MLP Policy
learning_rate	3e-4
buffer_size	300 000
batch_size	512
averaging_success_window	10
train_freq	1
tau	1e-3
gamma	0.99
gradient_steps	1
ent_coef	"auto"
target_entropy	"auto"
use_sde	False
net_arch	actor: [512, 1024, 512], critic: backbone [512, 1024, 512], heads: [512, 256]
log_std_init	-3
use_expln	False